RISC-V Processors: Performance and Cache Configurations

Justin Hsu, Dorian Chiu, Zifeng Yu, Tyler Wong
Department of Electrical Engineering
University of California
Davis, USA

Abstract - Cache performance is integral to high performance processors by supplying quickly accessible, frequently used data for programs. This project presents an analysis of hardware performance based on different cache configurations for a CVA6 processor generated using Chipyard. Chipyard is a tool that supports agile hardware development-allowing for the quick generation, simulation, and physical layout of various processor IPs. Due to Chipyard's highly parametrizable nature, IP customization is very easy to implement, such as adjusting core count, using larger caches, and integrating other IPs such as accelerators. By simulating different cache designs across both the instruction and data cache, performance is analyzed through a series of different software tests, each to highlight a different function the cache must adequately perform. D-Cache associativity slightly reduces conflict misses, but yields diminishing returns in overall execution time. Increasing the cache block size (line width) drastically reduces miss rates, but actively degrades performance due to memory saturation. Furthermore, the processor area, power consumption, and leakage is also analyzed based on the different cache configurations.

Keywords: RISC-V, Cache, CVA6, Chipyard

I. Introduction

Every generation of new processors innovate on the previous generation through additional parallelism via additional cores, improvements to functional units, and other hardware optimizations. However, due to the inherent limits of computing, those processors must be fed with sufficient instructions and data to process, otherwise a stalled processor waiting for work does not achieve the performance results expected for a new generation of high performance processors. Due to this, cache design is becoming increasingly more critical, as designers have to balance the tradeoff between large caches with sufficient capacity to hold relevant data, but small enough that the area, power, and leakage of the cache does not dominate the entire design. The memory wall highlights the difference in performance between memory bandwidth and latency. Larger caches are becoming increasingly prevalent as main memory accesses are to be avoided so as to not suffer the access time penalty, yet must be carefully designed as to not degrade system performance through excessive area usage or leakage.

Caches are small SRAM arrays located on chip next to the logic of the processor-lending to their extremely fast access times compared with main memory. Caches utilize both spatial and temporal locality of programs-taking advantage of the repeated use of data, as well as the high likelihood of neighboring data being used. By storing this data within the cache which the processor can quickly access, processor stalls are avoided as the processor does not have to wait for the data to be retrieved by the slow main memory. As such, program latency is minimized, and performance is maximized as the processor can spend more time computing rather than waiting for data. Furthermore, cache hierarchies are used to make up for the limited capacity of L1 caches. Through a system of larger and larger caches (albeit with slower access times than L1 cache, but still significantly

faster than main memory), higher level caches can make up for the limited capacity of the lowest level cache, acting as a supplemental storage for when L1 cache is at maximum capacity.

Cache organization is incredibly important as well. Due to the limited nature of cache capacity, data must be mapped to storage spots in the cache in a logical manner, otherwise data aliasing and conflict errors occur, degrading the performance of the cache. Our analysis focused on adjusting both cache line width and cache associativity. Cache line width takes advantage of spatial locality-data gets pulled into cache from main memory as entire rows of data, so the wider the cache line width the more adjacent data can be pulled at once. Thus, the more a program takes advantage of spatial locality, the more a wider cache line width will benefit from. However, this also places more stress on the cache itself in the form of false sharing. In multicore systems if only a small chunk of data within that wide line is edited, the entire line must be rewritten into memory. Additionally, false misses may manifest in multicore systems as well-if two cores require different data in two different spots within the same line, this line of data must be bounced around the two processors, effectively killing the speed and dramatically degrading the performance improvement from parallel computing. The other cache configuration we chose to focus on was cache associativity. Cache associativity describes the number of "ways" the cache contains. A way is a location within the cache that data gets mapped to. As such, associativity is a spectrum, from directly associative caches to fully associative caches. A directly associative cache maps data to one specific location within the cache. This allows for low power cache operation, as the processor only has to search that one specific location the data is mapped to. However, conflict misses have a high potential to manifest, for example if two pieces of data are mapped to the same location they will be constantly evicting each other. As a result, cache performance drops as the data has to be constantly pulled from main memory in the cache, only to be evicted as another piece of data is mapped to the same location, a process known as thrashing. Furthermore, directly associative caches are inefficient-even if the rest of the cache is empty, since data is mapped to only one location that data is forced to be stored at that location, despite the rest of the cache being empty. At the other end of the spectrum is a fully associative cache. The entire cache is one giant set-data can be stored anywhere within the cache. There are no conflict misses, since data can now be mapped to anywhere within the cache, but fully associative caches results in very expensive hardware both in terms of power, and area. Many comparators needed to compare the address against every single line in the cache simultaneously to find data. The additional hardware complexity from the comparators greatly increases area usage (thus decreasing the amount of available space for the SRAM data cells), but also consumes a massive amount of energy to not only power the comparators but to also search the entire cache. As a result, typically N-way associative caches are the sweetspot, balancing the flexibility of data storage a fully associative cache provides, while also enjoying the energy benefits of a directly associative cache. By having multiple sets, data can be stored in different locations within the cache, potentially alleviating conflict errors and the thrashing problems of directly associative caches. Furthermore, by having a limited amount of sets, the processor only needs to search in so many cache locations, decreasing the power consumption of the cache.

II. CVA6 Core

Using Chipyard [3], nine CVA6 cores were generated with different cache configurations. The CVA6 core, previously known as Ariane [1], is a tapeout proven core designed at ETH Zurich, and is available as an IP within Chipyard. CVA6 is a single in-order issue, in-order RISC-V core capable of booting Linux. It features a six stage pipeline and supports either the RV64GC or RV32GC RISC-V ISA. CVA6 was chosen due to its support for simulation with additional tools, and simplicity in its single issue in-order execution, and simple pipeline.

CVA6's six stage pipeline [1][2] includes the following stages: PC Gen, fetch, decode, issue, execute, and writeback. The pipeline is split into four parts: the frontend, in-order issue, execute, and commit. The frontend comprises of the PC Gen and fetch stages. CVA6 uses dynamic branch prediction to speculatively fetch instructions, and uses both a branch history table and a branch target buffer to minimize branching related stalls. Even though the processor is in-order, this dynamic branch prediction allows the CVA6 core to amortize latency by precomputing speculative branch instructions, then committing those instructions in order. Typically, for out-of-order speculative execution a reorder buffer is used (to support the highly parallel, high throughput out-of-order nature of superscalar processors), but due to the CVA6 core's simpler single issue in-order design, a scoreboard is sufficient. The scoreboard keeps track of the status of all issued instructions, including speculative ones. Furthermore, the scoreboard ensures in-order commit, as the speculative results are held within the scoreboard and only permitted to commit once the speculative instruction reaches the head of the commit queue and is verified as non-speculative. This way, CVA6 can still execute out-of-order, but ensure in-order commits while being able to recover from mispredictions by flushing the scoreboard.

CVA6 has a private L1 instruction and data cache (I-Cache and D-Cache respectively), vastly beneficial for our analysis as we only need to analyze one set of caches rather than an entire cache hierarchy across multiple caches and deal with cache coherency issues. Our data cache byte size was 32768 bytes, while our instruction cache byte size was 16384 bytes. The baseline cache configuration was a set associativity of four for the instruction cache, and a set associativity of eight for the data cache, with both line widths at 128 bits. This cache configuration served as the control, and for the other configurations associativity across both the instruction and data cache were changed, along with the line widths. The purpose of Instruction cache is to store instructions and reduce the miss rate. I-Caches are exceptionally predictable and work best with sequential patterned threads. Data cache is responsible for fetching data from the main memory and storing the data temporarily. D-Cache access patterns are unpredictable since a program could read a variable from one memory address, then access another memory address to read an array and update a counter. Figure 1 is a summary of all the different cache configurations.

ICacheByteSize = 16384;
DCacheByteSize = 32768;

Cache configurations

Name	I cache assoc	D cache assoc	I line width	D line width
cv32a6_imac_sv32	4	8	128	128
cv32a6_imac_sv32_lowerI	2	8	128	128
cv32a6_imac_sv32_higherI	8	8	128	128
cv32a6_imac_sv32_lowerD	4	4	128	128
cv32a6_imac_sv32_higherD	4	16	128	128
cv32a6_imac_sv32_longerI	4	8	256	128
cv32a6_imac_sv32_shorterI	4	8	64	128
cv32a6_imac_sv32_longerD	4	8	128	256
cv32a6_imac_sv32_shorterD	4	8	128	64

Figure 1. Cache configurations for CVA6

Through the generation of these different CVA6 generations with Chipyard, we were able to

analyze the performance of the processor with several programs each utilizing different aspects of data fetching to thoroughly test how different cache configurations would affect performance.

III. Tools Used

A. Chipyard

This project focuses on using Chipyard to generate a proven processor with different cache configurations [3]. Due to the tapeout readiness of the CVA6 processor generated from Chipyard, we are confident in the accuracy of the performance simulations with the different cache configurations. Chipyard is an RTL to GDSII ecosystem developed at UC Berkeley. It is a framework composed of RTL generators, VLSI physical design flow, and simulators to verify the functionality of designs. It uses Chisel as its HDL, enabling the high parameterization of designs, and Scala for ease of configuration. This project focuses solely on the RTL generation portion of Chipyard. Chipyard has many available tapeout proven, academic IPs available for implementation, including CPUs, various hardware accelerators, and network on chip (NoC) generators. Since Chipyard uses Chisel HDL, this allows for fine grain customization of each of the generated IPs, allowing for easy configuration such as the type and amount of cores instantiated (big or little core types, amount of each core), instantiating different cores within the same design, custom cache configurations (adjusting cache capacity, associativity, line width, etc.), and many more customization options. All the processor files are then generated as a collection of RTL files (in our project, a collection of SystemVerilog files), and available for the user. In this project, we used Chipyard to generate nine CVA6 cores with different cache configurations. The configuration for each core was written using Scala, and each customization option is defined as a function within Chipyard's codebase, allowing for ease of customization using only a few lines of code.

B. Design Compiler

Our project not only measured the performance of the different cache configurations, but also the power and area of the different caches. To measure the power and area of the CVA6 core itself, Synopsys Design Compiler was used to synthesize the logic portions of the CVA6 core using the 45nm Nangate PDK [4]. Cache and other memory cells were omitted due to not having the available macros to support synthesis of these cells. As a result, these cells were left as black box components within the design, and measured with Cacti7 instead.

To synthesize the design, all relevant SystemVerilog files were uploaded into a folder, and a custom makefile was written to compile the design using Design Compiler. The clock frequency was specified for 250MHz, well below the rated operating frequency of 900MHz [1], and the low effort compile option was used. Because this project focuses solely on virtual simulation, and there are no intentions for further work involving physical layout and tapeout, low effort compile offers a sufficient estimation for area and power while maintaining a balance of a reasonable synthesis time.

C. Cacti7

Cacti7 [5] was used for power and area estimation of the caches itself, all using the 45nm process node. Due to not having the cache and memory macros necessary for Design Compiler to synthesize, Cacti7 must be used instead to prevent Design Compiler from falling into an infinite loop of trying and failing to synthesize SRAM arrays from standard cells.

Cacti7 is a powerful tool capable of providing accurate cache area and leakage estimates.

IV. Performance

To determine the performance of the CPU cores and their configurations, each core was simulated and tested by running complex programs. The results are then analyzed on GTKWave which serves as a waveform visualizer and is a tool to analyze inputs, outputs, and measures latency and timing. Our group conducted four testbenches; matrix multiplication, stream (cache throughput) testbenches, stride (spatial locality) testbenches, and pointer (latency and associativity) testbenches. Matrix multiplication is a complex algorithm to compute gradients especially within GPUs, computer vision, and machine learning models. Matrix multiplication is a testbench that covers a variety of performance metrics. It can test the parallelism of the CPU due to the fact that matrix multiplication operations are not dependent on one another and that they can simultaneously without overlapping. The nature of matrix multiplication requires the program to fetch data from a particular memory address multiple times vigorously. Matrix multiplication excels especially when the core is optimized to predict the operation's sequential pattern and pull data before the program asks for it. Stream is designed to test the maximum throughput of data through the core's pipeline. The methodology behind stream is to make an oversized array, approximately four times the cache size. This is to force the cache to test the core's reaction when it reads an array that exceeds its cache size. Forcing the core to evict previous memory blocks. Another way stream tests the CPUs is by running four specific vector loops; testing raw data transfer via copy, simple ALU use while transferring via scaling, reading from two memory locations via arithmetic add or subtract, and advanced computation via scaling, addition, and reading multiple memory locations. By testing these specific instructions and loops, this testbench is able to detect where bottlenecks occur, since these ALU functions are very fast, if the cache throughput is slow, ALUs will sit idle while waiting for data to arrive. Thus, it determines whether the CPU suffers from being memory bounded by the speed of the cache subsystem. Furthermore, stream is also used to test the write policies of the cache. D-Cache is set up to have a write through and a write back cache. Thus, stream exposes a traffic jam if the cache throughput is not strong enough causing the bus to be saturated with write signals affecting bandwidth negatively. This is because the purpose of stream is to run massive arrays to push the limits of the CPU. This testbench aims to test the spatial locality of the CPU core by fetching data N bits. If the stride is bigger than the cache's size, the cache will start to evict memory blocks causing more conflict misses. This benchmark is suitable for this experiment to test functionality of the branch predictors and how the CPU reacts when there is a gap within memory fetching. Lastly, pointer tests for latency and associativity based on pushing the limits of the cache. Pointer testbenches typically include recursive test cases in order to force the processor to complete their operation before moving to the next. The CPU is unable to move on to the next instruction without facing a serial dependency which is a metric to determine a core's latency. Additionally, pointer tests the associativity by forcing an eviction of memory blocks. These four particular testbenches simulate common complex algorithms that many industry-level cores run as these operations are becoming more essential due to the rise of Artificial Intelligence (AI) accelerators and Graphics Processing Units (GPU) are in high demand. In order to quantify these metrics, the number of cache misses is counted and compared between the test cases. The baseline cache performance is in the Figure 2 below.

		Cores:	cv32a6_imac_sv32
matmul		matmul	
	fst	Runtime	25740 ns
		Issued Stalls	998384
		L1 I-Cache miss	24229
		L1 D-Cache miss	2067
stream		stream	
	fst	Runtime	10047 ns
		Issued Stalls	320843
		L1 I-Cache miss	4851
		L1 D-Cache miss	12508
stride		stride	
	fst	Runtime	7368 ns
		Issued Stalls	298218
		L1 I-Cache miss	3815
		L1 D-Cache miss	18572
pointer		pointer	
	fst	Runtime	4984 ns
		Issued Stalls	164521
		L1 I-Cache miss	3341
		L1 D-Cache miss	1780

Figure 2. Unmodified Baseline CVA6 Core Results

A. Changing Associativity

Our group expects the performance to improve with higher associativity. The simulations were run on one environment, RISC-V Proxy. This specific environment is used to support complex C-instructions [2]. RISC-V Proxy was unable to simulate the cores with lower associativity potentially to the compiler attempting to optimize the source code. Below are the results from running the core configurations on the RISC-V Proxy shown in Figure 3.

cores	cv32a6_imac_sv32_higherI	cv32a6_imac_sv32_higherD
matmul		
Runtime	25426 ns	25607 ns
Issued Stalls	997249	996380
L1 I-Cache miss	24231	24235
L1 D-Cache miss	2056	2037
stream		
Runtime	10028 ns	10048 ns
Issued Stalls	322264	320803
L1 I-Cache miss	4881	4851
L1 D-Cache miss	12533	12438
stride		
Runtime	7429 ns	7376 ns
Issued Stalls	298579	297655
L1 I-Cache miss	3845	3801
L1 D-Cache miss	18562	18685
pointer		
Runtime	5109 ns	4983 ns
Issued Stalls	167386	164503
L1 I-Cache miss	3364	3340
L1 D-Cache miss	1780	1767

Figure 3. Modified Associativity I-Cache and D-Cache Results

Based on our hypothesis, changing the associativity to be higher should show an increase in performance. Examining the results, increasing the associativity to an 8-way associative cache overall improved by 0.02% when comparing 36321 cache misses with the modified cache to 36236 baseline cache misses. On the other hand, increasing the D-Cache to a 16-way associative cache resulted in a net zero performance. The net performance from the baseline cache to the higher associative D-Cache are the same overall, but the individual tests yielded differing results. Only matrix multiplication saw an improvement of 1% in runtime while the other testbenches had diminishing returns of less than a +/- 0.1% improvement. This means that the type of testbench affects the total D-Cache misses, but yields a small change in performance to where it is negligible. However, the result shows that the effect is dependent on a different workload rather than a single uniform reason.

These results are preferable since increasing cache size often leads to higher hardware complexity and ensures that having the

default associativity can do complex algorithms at an industrial scale. Highlighting a key weakness of this method; hardware complexity tax. Reducing the power and area, but increasing the performance is the top goal in the industry, however, to increase associativity, this would mean wider multiplexers and massive parallel tag comparators which elongates the critical path and stall times. This explains why we only see a slight reduction in misses, but no improvement in faster execution times. In conclusion, we determined that the associativity of the core configurations were not challenged by the testbenches extensively. This could be due to the memory bits experiencing thrashing due to the memory being too wide for the testbench depth and due to hardware limitations. In the future implementing a higher stress test for associative cache should yield better results. Based on the inconsistency of executing core configurations with lower associativity, our group determined that they were invalid.

B. Modifying Block Size

As mentioned before changing the block size (line width) allows the cache to fetch the entire block rather than a specific variable or instruction. Below in Figures 4 and 5, are the results of all simulations with runtime, issued stalls and I-Cache and D-Cache misses and total I-Cache cache misses compared between the three configurations.

		Cores:	cv32a6_imac_sv32_longerI	cv32a6_imac_sv32_shorterI	cv32a6_imac_sv32_longerD	cv32a6_imac_sv32_shorterD
matmul			matmul			
	fst	Runtime	25568 ns	25983 ns	25738 ns	25736 ns
		Issued Stalls	1000642	1011712	998518	997934
		L1 I-Cache miss	16729	46495	24245	24225
		L1 D-Cache miss	2053	2053	1073	4003
stream			stream			
	fst	Runtime	10154 ns	10559 ns	10057 ns	10247 ns
		Issued Stalls	324278	338458	320509	319813
		L1 I-Cache miss	3093	8423	4859	4870
		L1 D-Cache miss	12532	12516	6317	24835
stride			stride			
	fst	Runtime	7422 ns	7435 ns	7508 ns	7502 ns
		Issued Stalls	298543	299845	297946	298777
		L1 I-Cache miss	2359	6813	3838	3836
		L1 D-Cache miss	18575	18171	16097	23429
pointer			pointer			
	fst	Runtime	4967 ns	5159 ns	4991 ns	5253 ns
		Issued Stalls	165311	170916	164823	172017
		L1 I-Cache miss	2157	5794	3346	3347
		L1 D-Cache miss	1780	1782	913	3491

Figure 4. Summary of Modified I-Cache and D-Cache Block Size Data

Figure 5. Bar Chart of Total I-Cache Cache Misses compared between shorter, baseline, and longer block size

When investigating the I-Cache, the baseline had a total of 36236 cache misses combined. When the I-Cache block size was increased to 256 bits, there were a total of 24338 cache misses meaning a reduction of about 33% from the baseline cache misses. Compared to a shorter block size of 64 bits, this resulted in a total of 67525 cache misses, most notably when running matrix multiplication our group observed nearly a 92% increase in cache misses. On the other hand, below in Figure 6, are the total D-Cache cache misses compared to each other.

Figure 6. Bar Chart of Total I-Cache Cache Misses compared between shorter, baseline, and longer block size

By extending the D-Cache to make it longer to 256 bits, the D-cache cache misses are reduced by 30% when compared to the baseline with a total of 24400 D-Cache cache misses as compared to the baseline total of 32927. Matrix multiplication and stream test cases resulted in a 50% reduction in D-Cache misses which makes sense since the data is sequential and often fetching from the same memory block repeatedly. When shortening the D-Cache to 64 bits our simulation observed over a 60% increase in D-Cache cache misses. Matrix multiplication, stream testbench, and pointer testbench saw nearly an increase of 93%, 99%, and 96% in D-Cache cache misses. This is expected since the testbenches are stress testing the core to read beyond the memory block and read memory repetitively. Thus, fetching 64 bits would result in an increase in compulsory and capacity misses since there is less data that can be stored. Thus, evicting memory blocks and requiring more time to fetch. Overall, shortening the block size for I-Cache yielded approximately an 86% increase in cache misses. In conclusion, performance wise calculated by comparing the runtimes for each simulation at most yielded about a 5% change and on average about a 1.5% change in time. Most notably when running the pointer testbench, the core was 3.51% and 5.4% slower than the baseline when running with a shorter I-Cache and D-Cache line width respectively. There is no significant change when modifying the I-Cache and D-Cache and the unmodified baseline performance is better as noted when running stream and stride testbenches. Investigating to modify a longer I-Cache block would prove most useful as it had meaningful improvements by running 0.67% faster in matrix multiplication and 0.34% in the pointer testbench. D-Cache on the other hand, resulted in net performance loss and actively degrades execution speed.

V. Area and Power

The pure CVA6 logic synthesized by Design Compiler provided a rough estimation of $0.04581\ cm^2$ of are used. Sequential elements used $0.0283\ cm^2$ of the area, combinational elements used $0.0175\ cm^2$ of the area, and buffers and inverters used $0.00113\ cm^2$ of the area. Around two million standard cells were placed, broken down into 1.432 million combinational elements, and 621,000 sequential elements. The operating voltage of the core was set to 1.1v, while the operating frequency was set at 250MHz. As a result, the total dynamic power was estimated to be 90.74 mW, while the total leakage power was estimated to be 98.42 mW. The cell internal power was found to be 55.43 mW (61% of dynamic power), while the

net switching power was found to be 35.32 mW (39% of dynamic power). The clock network consumed 73.58 mW of total power, accounting for 41.59% of the design's total power. The current design meets slack, with 3.09 ns of positive slack, suggesting additional optimizations can be made to the synthesis of the core itself. This is to be expected, as the low effort compile option was used.

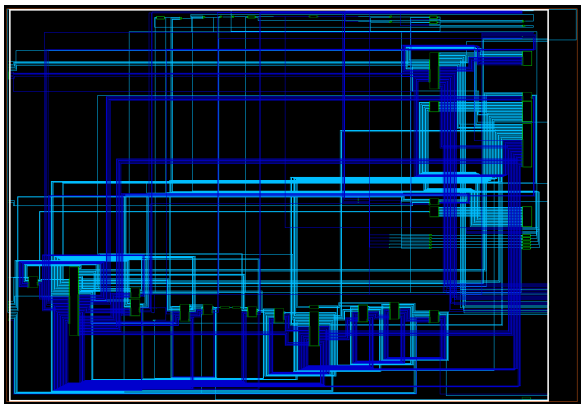

Figure 7. Synthesized CVA6 logic core

Further optimizations could include increasing the operating frequency to use up the available positive slack, or swap cells with high voltage threshold cells to decrease leakage and take advantage of the available positive slack.

Using Cacti7, all nine configurations of cache were computed using the 45 nm process node. The baseline data cache configuration of 8-way associativity with line width 128 had an overall area of 0.363 x 0.457 mm, with a leakage power of 54.41 mW and a maximum operational frequency of 2.14 GHz. The energy for each read access was 0.026 nJ while the energy for each write access was 0.036 nJ. The baseline instruction cache configuration of 4-way associativity with line width 128 had an overall area of 0.362 x 0.237 mm, with a leakage power of 28.64 mW and a maximum operational frequency of 3.03 GHz. The energy for each read access was 0.013 nJ while the energy for each write access was 0.023 nJ. These two baseline caches provided the control for the rest of our analysis, and was used as the benchmark for comparison. The power and area metrics of the other cache configurations are summarized in Figure 8.

Config Name	Cache Type	Size (KB)	Assoc	Block Size (B)	Access Time (ns)	Cycle Time (ns)	Max Freq (GHz)	Read Energy (nJ)	Write Energy (nJ)	Leakage (mW)	Total Area (mm²)	Data Area (mm²)	Tag Area (mm²)	Bounding Box (mm)
dcache_baseline	D	32	8	16	0.4246	0.4668	2.14	0.02592	0.03617	54.41	0.1366	0.1064	0.0302	0.363 x 0.457
icache_baseline	I	16	4	16	0.3772	0.3299	3.03	0.01339	0.02271	28.64	0.0704	0.055	0.0155	0.362 x 0.237
dcache_higherD	D	32	16	16	0.4839	0.8455	1.18	0.03392	0.03767	55.56	0.1427	0.1117	0.0311	0.198 x 0.733
icache_higherD	I	16	4	16	0.3772	0.3299	3.03	0.01339	0.02271	29.45	0.0856	0.055	0.0155	0.362 x 0.237
dcache_higherI	D	32	8	16	0.4246	0.4668	2.14	0.02592	0.03617	55.93	0.1863	0.1064	0.0302	0.363 x 0.457
icache_higherI	I	16	8	16	0.3957	0.4006	2.5	0.01726	0.03128	29.62	0.073	0.0571	0.0159	0.195 x 0.360
dcache_longerD	D	32	4	16	0.4246	0.4668	2.14	0.02607	0.0351	50.15	0.1371	0.1064	0.0302	0.363 x 0.457
icache_longerD	I	16	4	16	0.3772	0.3299	3.03	0.01339	0.02271	29.45	0.0856	0.055	0.0155	0.362 x 0.237
dcache_longerI	D	32	8	16	0.4246	0.4668	2.14	0.02592	0.03617	55.93	0.1863	0.1064	0.0302	0.363 x 0.457
icache_longerI	I	16	2	16	0.3385	0.4006	2.5	0.01527	0.01956	24.95	0.0733	0.055	0.0155	0.195 x 0.360
dcache_lowerD	D	32	16	16	0.4839	0.8455	1.18	0.03392	0.03767	55.56	0.1427	0.1117	0.0311	0.198 x 0.733
icache_lowerD	I	16	4	16	0.3772	0.3299	3.03	0.01339	0.02271	29.45	0.0856	0.055	0.0155	0.362 x 0.237
dcache_lowerI	D	32	8	16	0.4246	0.4668	2.14	0.02592	0.03617	55.93	0.1863	0.1064	0.0302	0.363 x 0.457
icache_lowerI	I	16	2	16	0.3754	0.3299	3.03	0.01264	0.01767	29	0.0849	0.055	0.0155	0.362 x 0.237
dcache_shorterD	D	32	8	32	0.5269	0.4668	2.14	0.02904	0.03461	67.98	0.1862	0.1064	0.0302	0.363 x 0.457
icache_shorterD	I	16	4	16	0.3772	0.3299	3.03	0.01339	0.02271	29.45	0.0856	0.055	0.0155	0.362 x 0.237
dcache_shorterI	D	32	8	16	0.4246	0.4668	2.14	0.02592	0.03617	55.93	0.1863	0.1064	0.0302	0.363 x 0.457
icache_shorterI	I	16	4	32	0.3911	0.3299	3.03	0.01642	0.02307	35.19	0.1147	0.055	0.0155	0.362 x 0.237

Figure 8. Cache power and area summary

There is a proportional relationship between the capacity of the cache, area, read/write energy, and leakage. Additional capacity requires more SRAM cells, meaning additional transistor devices are necessary to implement those cells. In turn, more devices results in more energy needed to access those cells as there are now more cells in each column and row, and more transistor devices not only means more dynamic power consumption, but also more leakage power consumption as there are now more devices to leak current from. This power and area metric is yet another tradeoff cache designers must consider when designing caches for high performance systems. Lastly, some caches were synthesized to be more rectangular than square. For example Dcache_higherD, with a bounding box of 0.198x0.733 mm, suggesting a very non-uniform, elongated design. As a result, this particular cache has a very slow operating frequency of 1.18 GHz. The non-uniform nature of this design means critical paths of signals are not balanced (since this design is taller than it is long), and since operating frequency must at least support the slowest critical path, the elongated shape of this cache means a longer, unbalanced critical path, resulting in a much lower operational frequency. The rectangular nature of this cache design can be potentially explained by the increased associativity of 16, necessitating more vertically stacked sets that are skinnier due to less data needed to store in that set. Furthermore, the

relatively slower operating frequency can also be explained by all the comparisons required to ensure the tag comparisons and data muxing.

VI. Conclusions and Future Work

Although the current study provides a clear cache trade-off for CVA 6, several extensions are still needed to strengthen the conclusions.

First, the lower-associativity config should be revalidated under a fully controlled software flow. In our experiments, some lowerI and lowerD cases didn't execute reliably, and the observed failures appear to be related not only to cache structure, but also to software stack compatibility, including the RISC-V proxy kernel and compile flow. Attempting to run the testbenches with the core on baremetal yielded unusual results. Because of this, these configurations should be treated as unstable cases rather than direct architectural conclusions. In the future we aim to get results that exhibit challenging associativity.

Second, future work should separate architectural effects from toolchain effects more carefully. This includes rebuilding the workloads with a unified compiler configuration, validating proxy-kernel compatibility across all cache variants, and rerunning the failed cases to determine whether the errors are caused by cache behaviour, software assumptions or integration issues between the two.

Third, the performance study can be expanded with more workload classifications, especially branch-heavy kernels and mixed compute-memory programs, to better isolate frontend sensitivity, memory-side stall behaviour, and cache-line utilization effects. This would help explain why some cache changes did not match the originally expected trend.

Finally, this work could be extended with FPGA based or board level validation. Combining RTL simulation results, cache macro estimation, and hardware-backed measurement would provide a more comprehensive understanding of the tradeoff between performance, area, energy, and implementation practically.

References

[1] "CVA6 requirement specification," CVA6 Requirement Specification - CVA6 documentation, https://docs.openhwgroup.org/projects/cva6-user-manual/02_cva6_requirements/cva6_requirements_specification.html (accessed Mar. 13, 2026).

[2] OpenHW Group, "CVA6 RISC-V CPU Core Repository," GitHub. [Online]. Available: https://github.com/openhwgroup/cva6
OpenHW Group "CVA6 Design Document — CVA6 documentation," docs.openhwgroup.org. [Online]. Available: https://docs.openhwgroup.org/projects/cva6-user-manual/03_cva6_design/index.html

[3] UC Berkeley Architecture Research, "Chipyard Framework: An Integrated RISC-V SoC Design Environment," Documentation. [Online]. Available: https://chipyard.readthedocs.io/

[4] Synopsys, Inc., "Design Compiler: RTL Synthesis Solution," Product Page. [Online]. Available: https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler.html

[5] Hewlett Packard Labs, "CACTI: An Integrated Cache and Memory Hierarchy Design Tool," GitHub Repository. [Online]. Available: https://github.com/HewlettPackard/cacti

[6] A. Bybell, "GTKWave: A Fully Featured GTK+ Based Waveform Viewer," SourceForge. [Online]. Available: https://gtkwave.sourceforge.net/